

# Template Method Pattern

Prepared By: Team 2

Instructor: Md Samsuddoha

# The Macro View: Enter the Template Method

Behavioral Pattern

## The Template Method (Algorithm Skeleton)



**The Transition:** Many architectures begin with a Factory Method. In fact, a Factory Method often serves as just one specialized step within a much larger architectural structure: **The Template Method**.

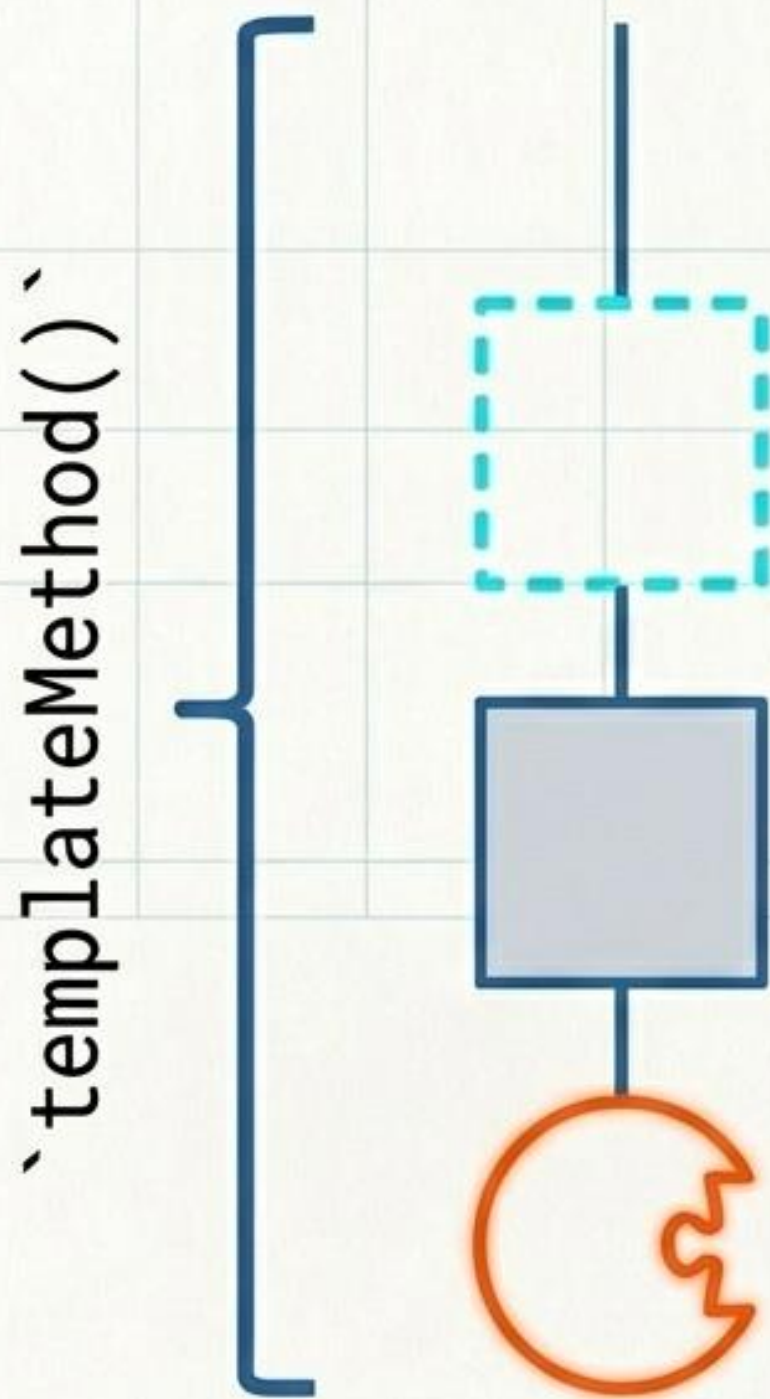
**Intent:** While Factory delegates *creation*, the Template Method defines the rigid skeleton of an entire *algorithm* in the superclass, letting subclasses override specific internal steps without ever changing the algorithm's overarching structure.

# The Dilemma: Redundant Algorithms



- **The Context:** Building a data mining app that extracts and normalizes corporate data from DOC, CSV, and PDF formats.
- **The Dilemma:** Extraction requires unique logic, but the subsequent steps—analyzing raw data and composing reports—are exactly the same across all three classes.
- **The Result:** Widespread duplication. Modifying analysis logic requires “shotgun surgery” across multiple files, and client code is choked with conditionals.

# The Solution: The Algorithmic Skeleton



**The Paradigm Shift:** Break the monolithic algorithm into steps. Centralize the sequence inside a single, strict *template method*.

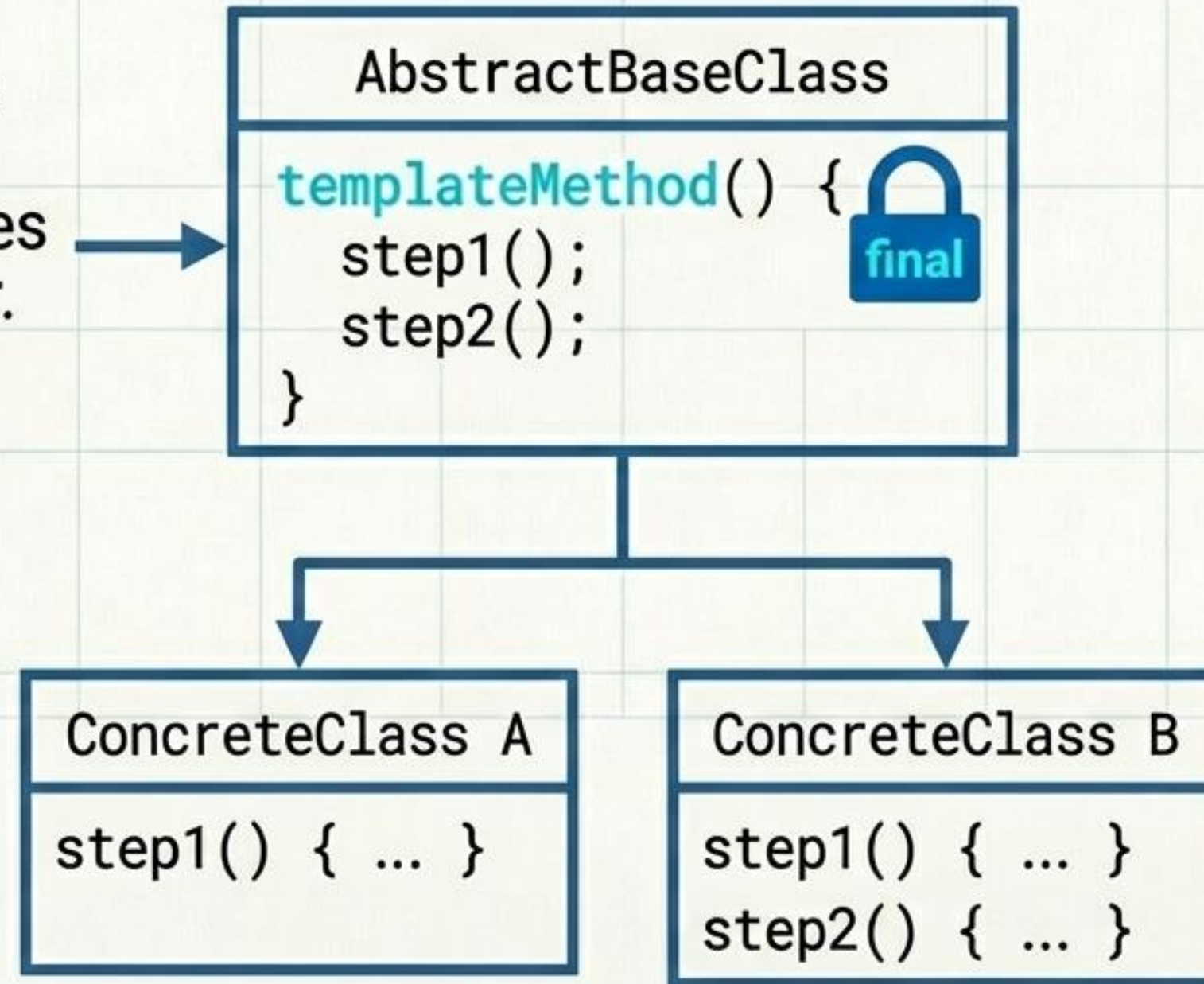
**Abstract Steps:** Have no implementation. Subclasses *must* implement them (e.g., parsing a specific file type).

**Optional Steps:** Have a default implementation in the base class. Subclasses *can* override them if needed.

**Hooks:** Optional steps with an empty body, placed before/after crucial points to provide subclasses a safe, isolated extension point.

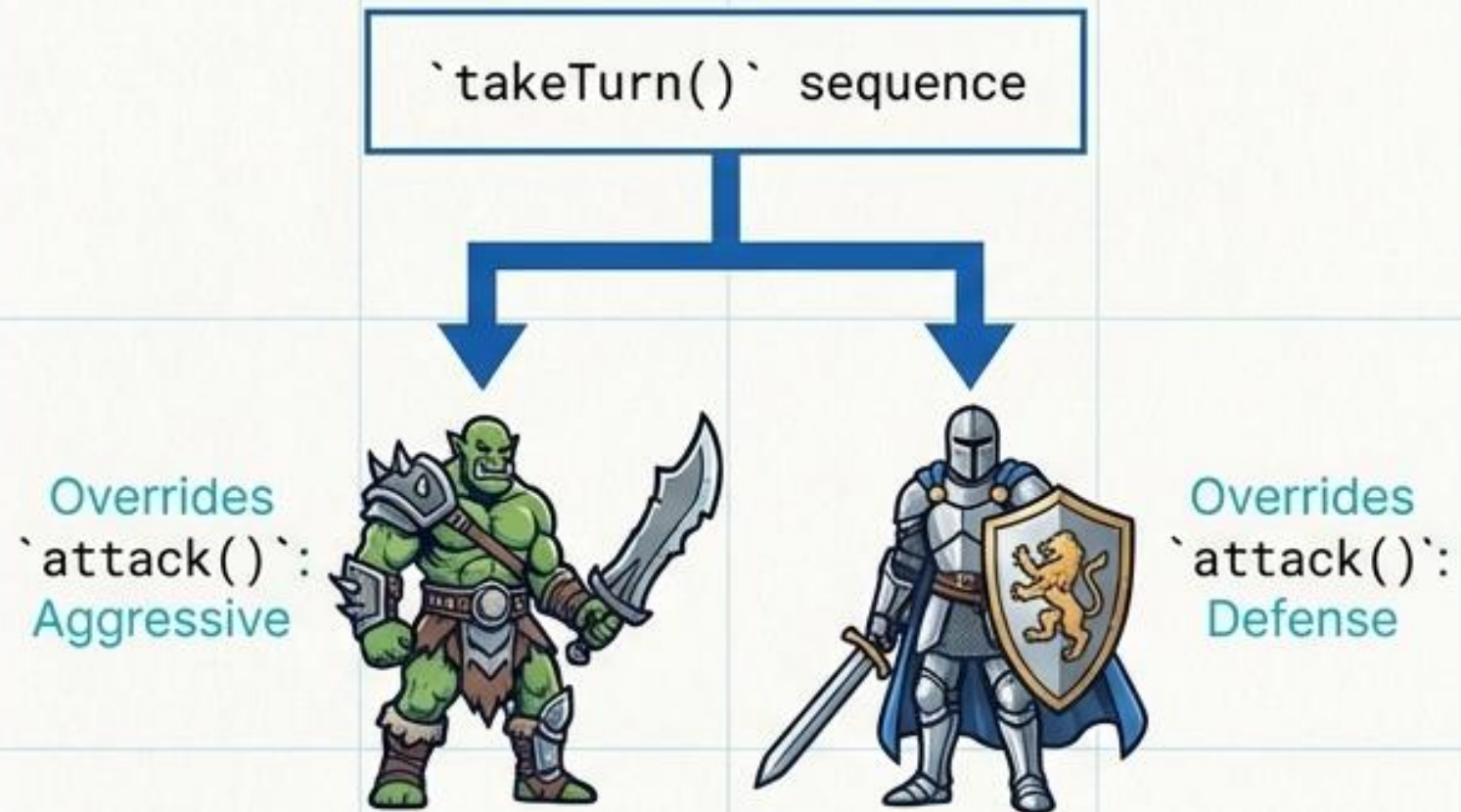
# Anatomy of the Template Method

**Abstract Base Class:**  
Houses the template method which dictates strict execution order. Marked `final` to mathematically prevent hijacking the sequence.



**Concrete Classes:**  
Inherit the skeleton. Provide specific implementations for abstract steps. They *\*cannot* alter the overarching template method.

# Analogies: Game AI and Mass Housing



**Software Example:** Strategy AI. A base `'AI'` class defines a strict turn sequence (gather, build, attack). The Orc subclass overrides `'attack()'` to be aggressive; Human focuses on defense. The turn sequence remains untouched.



**Real-World Analogy:** Mass housing. The architectural template dictates the foundation and framing sequence. Subclasses (homeowners) utilize hooks to alter specific details (paint, windows) making the house unique without compromising structural integrity.

# Architectural Trade-offs: Template Method



## Advantages

1. **Isolation:** Clients can override only specific parts of a massive algorithm, shielding them from upstream changes in other steps.

2. **DRY Principle:** Pulls identical code (like data analysis logic) up into a superclass, completely eliminating duplication.



## Disadvantages

1. **Rigidity:** Clients are strictly limited by the skeleton provided by the superclass.

2. **Liskov Substitution Principle (LSP):** High risk of violating LSP if a subclass improperly suppresses or alters the intent of a default step implementation.

3. **Maintenance Burden:** Template methods become increasingly difficult to maintain as the number of sequential steps grows.

# When and How to Implement

```
abstract class DataMiner {  
    // The Template Method  
    final void mine(String path) {  
        openFile(path);  
        extractData();  
        analyzeData();  
        closeHook();  
    }  
  
    // Abstract, Default, and Hook steps  
    abstract void extractData();  
    void analyzeData() { /* default */ }  
    void closeHook() { /* empty */ }  
}
```

## When to Use

- To let clients extend specific steps of an algorithm without breaking the core structure.
- To consolidate multiple classes containing almost identical algorithms, preventing widespread 'shotgun surgery' during updates.

## Implementation Logic

1. Break target algorithm into sequential steps.
2. Create abstract base class with a `final` template method.
3. Outline the execution sequence within the method.
4. Declare step methods as `abstract` or provide defaults.
5. Add empty `hook()` methods between crucial steps.

Any questions?